

Desenvolvimento Web

Guia de Estudos — Parte 3: Enviando dados para a API (POST, PUT e DELETE)

No guia anterior você aprendeu a **ler** dados de uma API com `GET`. Agora vamos aprender a **criar**, **atualizar** e **apagar** dados — fechando o ciclo completo de um app de verdade. Todo o material usa o projeto **TaskFlow**.

Projeto: TaskFlow · API: JSONPlaceholder

Sumário

1. Relembrando: os quatro verbos de uma API (CRUD)
2. A grande ideia: o `fetch` com opções
3. POST — criar uma tarefa nova
4. PUT — atualizar uma tarefa que já existe
5. DELETE — remover uma tarefa
6. A "pegadinha" do JSONPlaceholder
7. Revisão para a prova — checklist

1. Relembrando: os quatro verbos de uma API (CRUD)

Toda aplicação que trabalha com dados faz basicamente quatro coisas: **criar**, **ler**, **atualizar** e **apagar**. Em inglês, as iniciais formam a sigla **CRUD**. Na web, cada uma dessas ações tem um **verbo HTTP** correspondente que dizemos ao servidor:

Ação (CRUD)	Verbo HTTP	O que faz	Você já viu?
Read (ler)	GET	Busca / lê dados	Sim, no guia anterior
Create (criar)	POST	Envia um dado novo	É a aula de hoje
Update (atualizar)	PUT	Substitui um dado existente	É a aula de hoje
Delete (apagar)	DELETE	Remove um dado	É a aula de hoje

Analogia — o almoxarifado com um atendente

Imagine que o servidor é um **almoxarifado** e você (o app) só consegue falar com ele mandando **bilhetes padronizados** ao atendente:

- **GET** — "me mostra o que tem na prateleira 5" (só olha, não muda nada).
- **POST** — "guarde este *item novo* numa prateleira" (cria algo que não existia).
- **PUT** — "troque o conteúdo da prateleira 5 por *este aqui*" (substitui o que já estava lá).
- **DELETE** — "esvazie a prateleira 5" (apaga o que estava lá).

Repare: no GET você não entrega nada ao atendente. Nos outros três, você entrega um bilhete **com o item junto** (POST e PUT) ou **dizendo qual prateleira mexer** (PUT e DELETE).

2. A grande ideia: o `fetch` com opções

No GET, usávamos o `fetch` só com o endereço:

```
await fetch("https://jsonplaceholder.typicode.com/posts");
```

Para os verbos que **mudam** dados, o `fetch` recebe um **segundo argumento**: um objeto de **opções**. É aí que dizemos qual verbo usar e, quando necessário, qual dado enviar.

```
await fetch(url, {
  method: "POST", // o verbo
  headers: { "Content-Type": "application/json" }, // aviso: estou mandando JSON
  body: JSON.stringify(objeto), // o dado, virado texto
});
```

São três chaves importantes:

Chave	Para que serve	Quem usa
method	Diz o verbo: "POST" , "PUT" ou "DELETE"	Todos
headers	Avisa o servidor que o corpo é JSON	POST e PUT
body	O dado a enviar, convertido em texto com <code>JSON.stringify</code>	POST e PUT

Por que `JSON.stringify` ?

O `body` só viaja pela internet como **texto**. Nosso dado é um objeto JavaScript (`{ title: "...", body: "..."}`). O `JSON.stringify()` transforma esse objeto em uma **string** de texto para poder ser enviado. É o inverso do `.json()` que usávamos ao *receber* a resposta.

3. POST — criar uma tarefa nova

Quando o usuário preenche o formulário e clica em "**Adicionar tarefa**", o TaskFlow chama a função `handleCriar`, passando um objeto com `title` e `body`. Nosso trabalho é enviar esse objeto para a API e depois colocar a tarefa criada na tela.

src/App.jsx (editar a função `handleCriar`)

```
async function handleCriar(nova) {
  const resposta = await fetch(API, {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify(nova),
  });

  const tarefaCriada = await resposta.json(); // o servidor devolve a tarefa com um id

  setTarefas([tarefaCriada, ...tarefas]); // coloca a nova no início da lista
}
```

Lendo linha por linha

- `fetch(API, {...})` — manda o bilhete "crie isto" para `/posts`.
- `body: JSON.stringify(nova)` — o item que vai junto no bilhete.
- `await resposta.json()` — o servidor responde com a tarefa criada (agora com um `id`).
- `setTarefas([tarefaCriada, ...tarefas])` — cria um **novo array** com a tarefa nova na frente, seguida de todas as antigas (`...tarefas` é o "espalhar" que vimos em imutabilidade).

Imutabilidade de novo

Nunca faça `tarefas.push(tarefaCriada)`. O React só percebe a mudança quando você entrega um **array novo** pelo `setTarefas`. Por isso usamos `[novo, ...antigos]` em vez de empurrar dentro do array antigo.

4. PUT — atualizar uma tarefa que já existe

Ao clicar em "**Editar**" num card, o formulário é preenchido com aquela tarefa. Quando o usuário salva, chamamos `handleAtualizar`, que recebe o **id da tarefa** e **os dados novos**. A diferença para o POST: o endereço agora aponta para **uma tarefa específica** — `/posts/id`.

src/App.jsx (editar a função `handleAtualizar`)

```

async function handleAtualizar(id, dados) {
  await fetch(`${API}/${id}`, { // repare nas crases `` e no /${id}
    method: "PUT",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify(dados),
  });

  setTarefas(
    tarefas.map((t) => (t.id === id ? { ...t, ...dados } : t))
  );

  setTarefaEmEdicao(null); // fecha o modo de edição
}

```

Lendo linha por linha

- ``${API}/${id}`` — o endereço da tarefa que queremos trocar (ex.: `/posts/3`).
- `method: "PUT"` — o verbo de substituir.
- `tarefas.map(...)` — percorre a lista; para a tarefa cujo `id` bate, devolve uma cópia com os dados novos (`{ ...t, ...dados }`); as outras ficam iguais.

Cuidado com as crases (o erro clássico)

O endereço ``${API}/${id}`` usa **crase** (```), não aspas. Se você usar aspas normais — `"${API}/${id}"` — o `${id}` vai ser enviado como **texto literal**, e a API não vai achar a tarefa. Crase = o JavaScript troca `${...}` pelo valor de verdade.

5. DELETE — remover uma tarefa

Ao clicar em "Excluir", chamamos `handleExcluir` com o `id` da tarefa. É o mais simples dos três: não enviamos `body` (não há dado para mandar), só dizemos **qual** apagar pela URL.

src/App.jsx (editar a função `handleExcluir`)

```
async function handleExcluir(id) {
  if (!confirm("Deseja mesmo excluir esta tarefa?")) return;

  await fetch(`${API}/${id}`, {
    method: "DELETE",
  });

  setTarefas(tarefas.filter((t) => t.id !== id)); // mantém só quem NÃO é o id
  apagado
}
```

Lendo linha por linha

- `confirm(...)` — abre uma janelinha de confirmação; se o usuário cancelar, o `return` interrompe a função e nada é apagado.
- Sem `headers` e sem `body` — no DELETE não mandamos nenhum dado, só o verbo.
- `tarefas.filter((t) => t.id !== id)` — cria uma lista nova só com as tarefas cujo `id` é **diferente** do apagado. Ou seja, tira aquela da tela.

Padrão que se repete

Reparou? **POST** → `[novo, ...tarefas]`, **PUT** → `tarefas.map(...)`, **DELETE** → `tarefas.filter(...)`. São sempre as mesmas três operações de array para atualizar a tela depois da resposta do servidor. Decorou esse trio, dominou o CRUD.

6. A "pegadinha" do JSONPlaceholder

O JSONPlaceholder é uma API pública **de treino**. Ele aceita seus POST, PUT e DELETE e responde "deu certo" — mas **não salva nada de verdade**. Se você criar uma tarefa e depois recarregar a página (F5), ela some, porque o GET vai buscar de novo a lista original do servidor.

Isso é um problema? Não.

É por isso que, nas três funções, a gente atualiza o **estado local** (`setTarefas`) por conta própria, em vez de esperar o servidor guardar. Num API real (com banco de dados) o dado ficaria salvo, e o resto do código seria **exatamente igual**. O que muda é só o endereço da API.

Ideia-chave: **a resposta do servidor e o que aparece na tela** são duas coisas separadas — o React desenha a tela a partir do estado (`tarefas`), não diretamente da API.

7. Revisão para a prova — checklist

Antes da prova, garanta que você sabe fazer cada um destes itens **sem colar** (a consulta é permitida, mas quanto menos depender dela, mais rápido você termina):

Componentes e props

- Criar um componente que recebe **props** e usá-las no JSX (`{props.titulo}` ou desestruturando).
- Renderizar uma lista com `.map()` e a prop `key` .

Estado e renderização condicional

- Criar estado com `useState` e atualizá-lo por um evento (`onClick` , `onChange`).
- Mostrar/esconder algo com `{condicao && <elemento />}` ou `{condicao ? A : B}` .
- Formulário controlado: `value` + `onChange` ligados a um estado.

API — os quatro verbos

Verbo	Endereço	Tem body?	Atualiza a tela com
GET	<code>/posts</code>	Não	<code>setTarefas(dados)</code>
POST	<code>/posts</code>	Sim	<code>[novo, ...tarefas]</code>
PUT	<code>/posts/id</code>	Sim	<code>tarefas.map(...)</code>
DELETE	<code>/posts/id</code>	Não	<code>tarefas.filter(...)</code>

Erros que mais aparecem — confira antes de entregar

- URL com **aspas em vez de crase** quando tem `${id}` .
- Esquecer o `await` antes do `fetch` ou do `.json()` .
- Esquecer `JSON.stringify` no `body` (mandar o objeto cru).
- Usar `push` /mudar o array direto em vez de criar um novo com `set...` .
- Esquecer a prop `key` no `.map()` .

Como estudar com o TaskFlow

Abra o projeto, rode `npm install` e `npm run dev` , e implemente as três funções `TODO` do `App.jsx` por conta própria. Se travar, volte aqui na seção do verbo correspondente. Ao final, teste: criar, editar e excluir uma tarefa devem funcionar na tela.